Consensus Guide

Nakamoto consensus: the double-spend race, the arithmetic of confirmations, and the deployed most-work rule

m@rek.onl

Abstract

Assuming only the *Math Guide*'s probability foundations, this volume of *The Zcash Arboretum* constructs Nakamoto consensus from first principles and quantifies the security it buys. It states the block tree and the most-work rule as the protocol specification defines them and as the deployed zebra implements them; builds, on the *Math Guide*'s foundations, the race-specific probability the analysis needs (memoryless clocks, the biased random walk, the negative binomial law); develops the exact double-spending analysis of Rosenfeld's *Analysis of hashrate-based double-spending* — the catch-up theorem, the confirmation formula, and the economics of the attack — with every number recomputed by script and the results instantiated for Zcash's 75-second blocks; and closes with the deployed safety rails (per-block difficulty adjustment, reorg windows, coinbase maturity, wallet confirmation policy) that frame the model's assumptions. Where the specification, the paper, and the deployed nodes disagree, the divergence is stated.

Contents

1 Scope, sources, and the two double-spends

Every volume above this one takes a working ledger for granted: the *Ironwood Guide* proves what a shielded transaction — one whose amounts and addresses are cryptographically hidden — commits to, the *Wallet Guide* builds transactions, the *Sync Guide* fetches the chain, the *FlyClient Guide* verifies it succinctly. This volume documents the mechanism that makes there be *one* chain to fetch: Nakamoto consensus — proof of work, the most-work rule, and the probabilistic finality they purchase. The treatment is quantitative throughout. Its spine is Meni Rosenfeld's *Analysis of hashrate-based double-spending* (11 December 2012; latest version 12 February 2014; arXiv:1402.2009), the paper that replaced the approximate analysis in Satoshi Nakamoto's whitepaper with an exact one; we reprove its results in full, recompute its tables by script, and instantiate its conclusions for the deployed Zcash parameters.

Two distinct attacks are both called "double-spending", and this volume treats exactly one of them. Spending the same note twice *within* one chain is impossible in Zcash by construction: each spend reveals a nullifier, and a block that repeats a nullifier is invalid (*Ironwood Guide*, §"Prevention of double-spending: nullifiers"). What consensus must prevent is the chain-level attack: paying a merchant on the chain everyone sees, then replacing that chain wholesale with another in which the payment never happened. No zero-knowledge proof (*Crypto Guide*, §"Interactive proofs, zero knowledge, and SNARKs") rules this out; only the economics of proof of work bounds its probability, and this volume computes that bound.

The volume assumes exactly one thing from the series below it: the *Math Guide*'s probability section (§"Probability, asymptotics, and computation"), which constructs probability from its definition — finite spaces, conditioning, independence, random variables, expectation — and extends it to the three rungs used here: countable additivity, continuity along monotone events, and density-defined laws (§"Beyond finite spaces: countable additivity, limits, and densities"). On that base, Section 3 constructs the race-specific instruments no lower volume owns — memoryless clocks, random walks, the negative binomial law; the algebra used is elementary. Readers wanting only the results can read Sections 2, 6, and 8 and take the theorems on faith.

1.1 Sources and their standing

Table 1 lists the sources. The analysis (the paper and the whitepaper) is not normative for anything; the protocol specification and the deployed nodes are the ground truth for what Zcash actually does, and where the model's assumptions and the deployment diverge, Section 8 says so explicitly. Worked numbers throughout are computed by script (`.wip/consensus-drafts/compute.py`, whose output is the source of every numeric claim in this volume); the script reproduces both of the paper's printed tables cell-for-cell — Table 1 to its three-decimal rounding, Table 2 to its floored thousands-and-millions convention — before departing from them.

Source	Role	Pin
Rosenfeld, arXiv:1402.2009	primary analysis	v1, 2014-02-09
Nakamoto whitepaper, §11	historical analysis	2008
Zcash protocol specification	normative	`zips` `69610984`
ZIP-208 (Blossom)	normative	same pin
zebra	the deployed validator	`fa7657f1c`[1]
zcashd v6.10.0	retired validator	`16ac74376`
librustzcash	deployed wallet layer	`e30517e43`

Table 1: Sources. The paper is analysis, not specification: nothing in it binds the protocol. Deployed-behaviour claims cite crate, file, and line at the pinned commits. The retired zcashd is cited as history: its choices explain constants that survive it (Section 8.4).

Remark 1.1 (The paper's own condition)**.** The arXiv v1 PDF of the paper is lightly broken: it contains exactly two citations, both rendered as "[?]" (in the abstract and in §1), and no References section at all. Both plainly intend the Nakamoto whitepaper, and we read them so. The paper numbers only two of its displayed equations — its equation (1) is our Theorem 5.2, its equation (2) our equation (3) — and sets its two model assumptions as a bare numbered list; the *Assumption* environments of Section 4 are ours.

2 The block tree and the most-work rule

Zcash transactions are grouped into blocks, and every block names its parent by including the parent header's hash (*Crypto Guide*, §"Hash functions and the random oracle model"); the genesis block alone has no parent. Blocks therefore form a tree rooted at genesis, and each root-to-leaf path — each *branch* — is one internally consistent version of history. Branches need not be consistent with one another: one branch can contain a transaction that conflicts with a transaction in a sibling branch, and that freedom is exactly the attack surface this volume quantifies.

2.1 Work, and the best chain

A single coherent history requires a rule every node can apply locally and still agree on. The rule is not "longest": it weighs each block by the computational effort its header proves.

Definition 2.1 (Work of a block)**.** Let ToTarget be the conversion from the header's `nBits` field to its 256-bit target threshold (specification §7.7.4). The *work* of a block is

$$\mathsf{work} \;=\; \left\lfloor \frac{2^{256}}{\mathsf{ToTarget}(\texttt{nBits}) + 1} \right\rfloor$$

(specification §7.7.5, "Definition of Work"). A lower target admits fewer valid headers, so a block meeting it certifies proportionally more expected hashing; work is that certificate made additive.

The deployed validator computes this quantity with a 256-bit in-word trick: zebra evaluates `(!expanded.0 / (expanded.0 + 1)) + 1` (`zebra-chain/src/work/difficulty.rs:401`, stored in a `u128`); the retired zcashd computed the identical expression on `arith_uint256`, with `~` spelling the complement (`src/pow.cpp:144--157`). The form agrees with Definition 2.1 exactly: writing t for the target and $A = 2^{256}$, the complement is $\neg t = A - 1 - t$, and

$$\left\lfloor \frac{A-1-t}{t+1} \right\rfloor + 1 = \left\lfloor \frac{A-(t+1)}{t+1} \right\rfloor + 1 = \left\lfloor \frac{A}{t+1} \right\rfloor,$$

so the in-word computation equals the out-of-word quotient (checked by script over random targets).

Definition 2.2 (Best valid block chain)**.** Specification §3.3 ("The Block Chain"): a node "sums the work, as defined in §7.7.5, of all blocks in each valid block chain, and considers the valid block chain with greatest total work to be best. To break ties between leaf blocks, a node will prefer the block that it received first."

Rosenfeld's paper states the same rule for Bitcoin — "the branch representing the most proof of work", with first-seen tie-breaking — so the model transfers without adjustment. The popular shorthand "longest chain" is a corruption: length and work coincide only while difficulty is constant, which is precisely the paper's Assumption 4.2 and not a property of the deployed chain (Section 8.2).

[1] A working-branch checkout; every file cited in this volume was verified byte-identical to `ZcashFoundation/zebra` main at `74cef5e6d`, so the citations hold for both.

Note also what the rule does *not* say: nothing marks a chain as permanently chosen. A node that learns of a heavier branch switches to it, however deep the fork point — subject only to the node-local rails of Section 8.4. Consensus is a standing competition, not an election with a winner.

Definition 2.3 (Confirmations)**.** A transaction has *n confirmations* if it is included in a block of the best chain and there are *n* blocks on the path from that block to the chain's leaf, inclusive. The block containing the transaction is its first confirmation.

Different nodes may briefly disagree on the best chain when two blocks of equal cumulative work arrive in different orders; the disagreement is resolved by the next block, which makes one branch strictly heavier (Figure 1). Such ties are the benign case. The attack of Section 5 is the malicious one: a fork manufactured in private and released only when it wins.

(a) tied branches: nodes disagree

(b) the next block breaks the tie for every node

Figure 1: Branch selection. Arrows point from child to parent. In panel (a) two blocks extend the same parent and the branches carry equal work; each node keeps the one it saw first. In panel (b) a new block lands on the upper branch, which now has strictly more cumulative work; all nodes converge on it and the lower block is orphaned. At constant difficulty "more work" and "longer" coincide, which is how the paper, and this volume's figures, may draw block counts.

2.2 The attack

The double-spend attack, as the paper lays it out (§3):

1. broadcast a transaction paying the merchant;
2. secretly mine a branch from the last pre-transaction block, containing instead a conflicting transaction paying the attacker himself;
3. wait until the merchant's transaction has *n* confirmations and the merchant, satisfied, hands over the goods;
4. keep extending the secret branch until it carries more work than the public one, then broadcast it. Every node switches to it, the payment to the merchant is no longer part of history, and the attacker has both the goods and the coins.

Nothing in step 4 violates any validity rule — the released branch is a perfectly well-formed chain, merely a different one. The only question is probabilistic: with what probability does the secret branch ever get ahead? Answering it exactly is the business of Sections 4 and 5; the toolkit comes first.

3 A probability toolkit

The analysis ahead needs four tools: memoryless clocks (to model block discovery), a reduction from continuous time to a coin sequence, the negative binomial distribution (to count the attacker's progress), and one normalisation lemma about races. Their foundations are the *Math Guide*'s: probability spaces, conditioning, independence, random variables, and expectation are constructed there from the definition of probability onward (§"Probability, asymptotics, and computation"), and the

extension beyond finite spaces — countable additivity, continuity along monotone events, density-defined laws, and infinite sequences of independent trials — is its §"Beyond finite spaces: countable additivity, limits, and densities". What no lower volume owns are the four instruments themselves; each is constructed here on that base, only as far as the critical path requires.

3.1 Memoryless clocks and the attribution sequence

Definition 3.1 (Exponential clock)**.** A random variable X is *exponential with rate* $\lambda > 0$ if $\Pr[X > t] = e^{-\lambda t}$ for all $t \geq 0$ — a law defined by the density $\lambda e^{-\lambda t}$ on $t \geq 0$ in the sense of the *Math Guide*'s density definition, with mean $1/\lambda$.

Lemma 3.2 (Memorylessness)**.** *An exponential X satisfies $\Pr[X > s + t \mid X > s] = \Pr[X > t]$ for all $s, t \geq 0$: having waited without success teaches nothing about the remaining wait.*

Proof. $\Pr[X > s + t \mid X > s] = e^{-\lambda(s+t)}/e^{-\lambda s} = e^{-\lambda t}$. □

Lemma 3.3 (Race of two clocks)**.** *Let X and Y be independent exponentials with rates λ and μ. Then*

$$\Pr[X < Y] \;=\; \frac{\lambda}{\lambda + \mu},$$

and the winning time $\min(X, Y)$ is exponential with rate $\lambda + \mu$.

Proof. Integrating over the time at which X fires,

$$\Pr[X < Y] = \int_0^\infty \lambda e^{-\lambda t} \Pr[Y > t]\, dt = \int_0^\infty \lambda e^{-(\lambda+\mu)t}\, dt = \frac{\lambda}{\lambda + \mu}.$$

For the minimum, $\Pr[\min(X, Y) > t] = \Pr[X > t]\Pr[Y > t] = e^{-(\lambda+\mu)t}$. □

Proposition 3.4 (The attribution sequence)**.** *Model the honest network and the attacker as independent exponential clocks with rates p/T_0 and q/T_0, each restarting when either fires (the model of Section 4). Then the sequence that records, for each successive block found by anyone,* who *found it, is an infinite sequence of independent trials in the* Math Guide*'s sense — an independent, identically distributed sequence of Bernoulli trials: each block is the attacker's with probability q and the honest network's with probability p, independently of all earlier attributions and of all elapsed times.*

Proof. By Lemma 3.3 the first block is the attacker's with probability $(q/T_0)/(p/T_0 + q/T_0) = q$. When a block is found, the finder's clock restarts by construction, and the loser's remaining wait is distributed as a fresh clock by Lemma 3.2; the state after each block is therefore probabilistically identical to the start, independent of the past. The claim follows by induction. □

Proposition 3.4 is the paper's bridge (its §3, with footnote 1 supplying the clock rates) from physical time to combinatorics: since the question "does the attacker ever get ahead?" concerns only the *order* of block discoveries, not their times, every result below is a statement about a p/q coin sequence, and the time constant T_0 disappears from the answers. Where the exponential model itself comes from — and how well Zcash's Equihash fits it — is a deployment question, taken up in Section 8.3.

3.2 The negative binomial law

Proposition 3.5 (Progress of the loser)**.** *In an attribution sequence with attacker probability $q = 1 - p$, let m be the number of attacker blocks found strictly before the honest network's n-th block. Then*

$$P(m) = \binom{m+n-1}{m} p^n q^m, \qquad m = 0, 1, 2, \ldots$$

— the negative binomial *distribution.*

Proof. The event fixes the first $m + n$ trials exactly this far: the $(m + n)$-th trial is honest (the n-th honest success), and among the first $m + n - 1$ trials exactly m are the attacker's, in any of $\binom{m+n-1}{m}$ orders. Each such order has probability $p^{n-1}q^m \cdot p$. □

Lemma 3.6 (First-to-n race)**.** *In the same sequence, let H_n be the event that the honest network reaches n blocks before the attacker does, and A_n the reverse. Then H_n and A_n partition the sample space, and*

$$\Pr[H_n] = \sum_{m=0}^{n-1} \binom{m+n-1}{m} p^n q^m, \qquad \Pr[A_n] = \sum_{h=0}^{n-1} \binom{h+n-1}{h} q^n p^h, \qquad \Pr[H_n] + \Pr[A_n] = 1.$$

Proof. Among the first $2n - 1$ trials one side already has n successes, so the race terminates; a tie is impossible because block counts advance one at a time. The event H_n says the n-th honest block arrives while the attacker holds $m \le n - 1$; summing Proposition 3.5 over those m gives the first sum, and A_n is the same computation with the roles of p and q swapped. □

Remark 3.7 (From hash trials to exponential clocks)**.** The exponential model is itself a limit, not an axiom. A miner's work is a stream of candidate evaluations, each an independent Bernoulli trial with a tiny success probability; the number of trials to the first success is geometric, and a geometric law with small success probability, viewed at the scale of its mean, converges to the exponential of Definition 3.1. The step that matters is *progress-freeness*: a candidate that fails must carry no information usable by the next one. Whether Zcash's Equihash satisfies this is examined with the deployed parameters in Section 8.3.

4 Playing catch-up

The attacker of Section 2.2 finds himself, at the moment the merchant ships, some number of blocks behind the public chain, and wins if his branch ever overtakes it. This section computes the probability of that event as a function of the deficit; the next section supplies the distribution of the deficit itself.

Both computations run under the paper's two standing assumptions, stated here nearly verbatim (the environment form is ours, Remark 1.1):

Assumption 4.1 (Constant hashrates)**.** The total hashrate of the honest network and the attacker is constant. Combined they have a hashrate of H, of which pH belongs to the honest network and qH to the attacker, where $p + q = 1$.

Assumption 4.2 (Constant difficulty)**.** The mining difficulty is constant, and such that with hashrate H the average time to find a block is T_0.

Under Assumption 4.2 every block carries equal work, so "most work" and "longest" coincide and the race can be scored in block counts. How far the deployed chain sits from these assumptions — difficulty adjusts after every block, hashrate drifts — is quantified in Section 8.2; neither gap changes the shape of the conclusions.

Definition 4.3 (The deficit walk)**.** Let z denote the honest chain's lead in blocks over the attacker's branch. By Proposition 3.4, each newly found block moves z up by 1 with probability p (honest block) or down by 1 with probability q (attacker block), independently of the past:

$$z_{i+1} \;=\; \begin{cases} z_i + 1 & \text{with probability } p, \\ z_i - 1 & \text{with probability } q. \end{cases}$$

The attack succeeds the moment z reaches -1: the secret branch is then strictly ahead, and releasing it rewrites history.

Theorem 4.4 (Catch-up probability; Rosenfeld §3)**.** *Let a_z be the probability that the walk of Definition 4.3, started at deficit z, ever reaches -1. Then*

$$a_z \;=\; \min(q/p,\,1)^{\max(z+1,\,0)} \;=\; \begin{cases} 1 & \text{if } z < 0 \text{ or } q \geq p, \\ (q/p)^{z+1} & \text{if } z \geq 0 \text{ and } q < p. \end{cases} \tag{1}$$

Proof. For $z < 0$ the walk has already succeeded, so $a_z = 1$; assume $z \geq 0$. Fix an integer $N > z$ and absorb the walk at both -1 and N; let $h_N(z)$ be the probability of reaching -1 before N. Conditioning on the first step,

$$h_N(z) = p\,h_N(z+1) + q\,h_N(z-1) \qquad (0 \leq z \leq N-1), \qquad h_N(-1) = 1, \quad h_N(N) = 0,$$

a second-order linear recurrence. The trial solution $h_N(z) = t^z$ satisfies it exactly when t is a root of the *characteristic polynomial* $pt^2 - t + q = p(t-1)(t-q/p)$.

If $q \neq p$ the roots 1 and q/p are distinct, so $h_N(z) = A + B(q/p)^z$; the boundary conditions give

$$h_N(z) \;=\; \frac{(q/p)^z - (q/p)^N}{(q/p)^{-1} - (q/p)^N}.$$

If $q = p$ the root is double and $h_N(z) = A + Bz$, giving $h_N(z) = (N-z)/(N+1)$.

Now let $N \to \infty$. The events $E_N = \{\text{reach } -1 \text{ before } N\}$ are nondecreasing in N, and their union is the event that the walk ever reaches -1: a walk that does so does it at a finite time, having visited finitely many states, so E_N holds for every N above the largest state visited. By continuity along monotone events (*Math Guide*, §"Beyond finite spaces: countable additivity, limits, and densities"), $a_z = \lim_{N\to\infty} h_N(z)$. For $q < p$ the term $(q/p)^N$ vanishes and the quotient tends to $(q/p)^{z+1}$; for $q = p$, $(N-z)/(N+1) \to 1$; for $q > p$, dividing numerator and denominator by $(q/p)^N$ sends both to -1, so the quotient tends to 1. □

Corollary 4.5 (Majority always catches up)**.** *When $q \geq p$, the attacker overtakes the public chain with probability 1 from any finite deficit. No confirmation count defends against a majority of the*

hashrate.

Corollary 4.6 (Geometric decay in the deficit)**.** *When $q < p$, each additional block of deficit multiplies the catch-up probability by q/p. Concretely (script-computed): at $q = 10\%$, $a_0 = 1/9 \approx 0.1111$ and $a_5 \approx 1.88 \times 10^{-6}$; at $q = 30\%$, $a_0 = 3/7 \approx 0.4286$ and $a_5 \approx 6.20 \times 10^{-3}$.*

Corollary 4.7 (Blocks, not time)**.** *Equation (1) contains neither T_0 nor any other time scale: the security of a deficit is a function of block* counts *alone. Two chains with different block spacings but the same q offer identical security per confirmation — and therefore different security per minute (Section 6.1).*

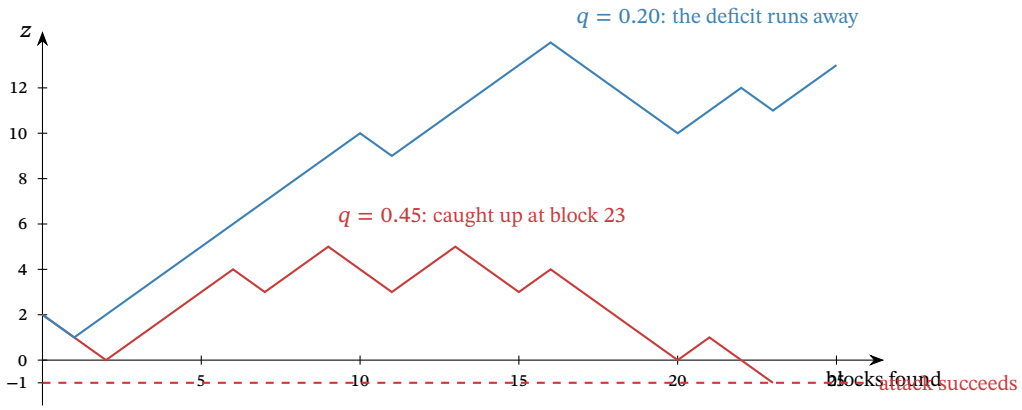

Figure 2: Two runs of the deficit walk from $z = 2$, simulated by script with fixed seeds. At $q = 0.45$ the drift is weak and this run reaches -1 after 23 blocks — the attack succeeds; at $q = 0.20$ the deficit grows roughly linearly, and by Theorem 4.4 the chance of ever returning from deficit 13 is $(1/4)^{14} \approx 3.7 \times 10^{-9}$.

5 Waiting for confirmations

The merchant's defence is patience: ship only after the paying transaction has n confirmations (Definition 2.3). This section computes exactly how much that patience buys. The paper's model of the attack timeline has one convention worth surfacing before the theorem.

Remark 5.1 (The pre-mined block)**.** The paper assumes the attacker banks one block before the race is scored: "we assume one block was pre-mined by the attacker before commencing the attack" (§4). The attacker forks from the public leaf at the moment the merchant's transaction is broadcast, and a patient attacker times the broadcast to coincide with a private block he has just found but not announced. At the moment the honest chain reaches n confirmations, the attacker who has found m further blocks therefore holds $m + 1$, and the deficit walk of Definition 4.3 starts at

$$z \;=\; n - m - 1.$$

The convention is a modelling choice, not a law: an attacker with no banked block starts at $z = n - m$, and every probability below shrinks accordingly. It is also, as Theorem 5.2 shows, precisely the choice that makes the final formula collapse into a symmetric race.

By Proposition 3.5, the number m of attacker blocks found while the honest network finds its n is negative binomial. Averaging the catch-up probability of Theorem 4.4 over m gives the central result.

Theorem 5.2 (Double-spend success probability; Rosenfeld eq. (1))**.** *Let the merchant wait for $n \geq 1$ confirmations, under Assumptions 4.1 and 4.2 and the convention of Remark 5.1. The probability that the attacker's branch eventually overtakes the public chain is $r(q, n) = 1$ if $q \geq p$, and otherwise*

$$r(q,n) \;=\; 1 - \sum_{m=0}^{n-1} \binom{m+n-1}{m} (p^n q^m - p^m q^n) \;=\; 2 \sum_{m=0}^{n-1} \binom{m+n-1}{m} p^m q^n. \tag{2}$$

Proof. Conditioning on m and applying Theorem 4.4 at $z = n - m - 1$,

$$r \;=\; \sum_{m=0}^{\infty} P(m)\, a_{n-m-1} \;=\; \underbrace{\sum_{m=0}^{n-1} P(m) \left(\frac{q}{p}\right)^{n-m}}_{\text{behind: must catch up}} \;+\; \underbrace{\sum_{m=n}^{\infty} P(m)}_{\text{already ahead}},$$

valid for $q < p$; when $q \geq p$ every $a_{n-m-1} = 1$ and $r = \sum_m P(m) = 1$ immediately. For the first sum, each term transforms exactly:

$$\binom{m+n-1}{m} p^n q^m \left(\frac{q}{p}\right)^{n-m} \;=\; \binom{m+n-1}{m} p^m q^n,$$

which by Lemma 3.6 is the probability that the attacker's n-th block arrives while the honest network holds m; summed over $m \leq n-1$, the first sum is $\Pr[A_n]$, the probability that the attacker wins a fair first-to-n race. The second sum is $\Pr[m \geq n] = 1 - \Pr[H_n] = \Pr[A_n]$ by the same lemma. Hence $r = 2\Pr[A_n]$, the right-hand form of (2); the middle form follows from $\Pr[A_n] = 1 - \Pr[H_n]$ applied to one of the two copies. The printed upper limit in the paper's equation (1) is n rather than $n-1$; the $m = n$ term is identically zero, so the forms agree. □

Remark 5.3 (The race doubling)**.** The proof exposes a structure the paper's algebra passes through silently: under the one-pre-mined-block convention, *the probability of falling behind and later catching up equals, exactly, the probability of never falling behind at all*, so the double-spend probability is twice the chance of winning a first-to-n race. The identity also absorbs the boundary case: at $q = p$ each race half is exactly $\frac{1}{2}$, giving $r = 1$ with no case split, continuously with the $q > p$ regime. The algebra here is the paper's; reading its two sums as the two halves of one race, and the observation that the pre-mine convention is what makes them equal, are this volume's presentational additions (verified by script over random (q, n); the two sums agree to floating-point precision).

Example 5.4 (A full race, exactly)**.** Take $q = 1/5$ (a twenty-percent attacker) and $n = 6$. The deficit starts at $z = 5 - m$, the catch-up factor is $(q/p)^{6-m} = 4^{-(6-m)}$, and every quantity below is an exact rational, printed to six decimals (script-computed):

m	$P(m)$	$a_{5-m} = 4^{-(6-m)}$	product
0	0.262144	0.000244	0.000064
1	0.314573	0.000977	0.000307
2	0.220201	0.003906	0.000860
3	0.117441	0.015625	0.001835
4	0.052848	0.062500	0.003303
5	0.021139	0.250000	0.005285
catch-up half (sum of products)			0.011654
already-ahead half, $\Pr[m \geq 6]$			0.011654
$r(0.2,\ 6)$			0.023308

The two halves agree to every printed digit — they are equal exactly, by Remark 5.3 — and the closed form (2) evaluates to the same 0.023308. A merchant facing a possible twenty-percent adversary who ships after six confirmations is running a 2.33% risk.

Remark 5.5 (Satoshi's approximation)**.** The whitepaper's §11 computes the same quantity with one approximation and two offsetting conventions: the attacker's progress is approximated as Poisson with mean $\lambda = nq/p$ (the law $\Pr[k] = e^{-\lambda}\lambda^k/k!$ on the nonnegative integers); no block is pre-mined, but success is scored at *breakeven* rather than strictly ahead, and those two conventions cancel exactly — the whitepaper's catch-up factor for m attacker blocks is $(q/p)^{n-m}$, term for term the factor of Remark 5.1. The Poisson approximation is the entire gap, and it pushes the estimate down. Script-computed comparisons: at $q = 10\%$, $n = 6$, the whitepaper's formula gives 0.0243% against the exact 0.0591% — an understatement by a factor of about 2.4; at $q = 30\%$, $n = 6$: 13.211% against 15.645%; at $q = 10\%$, $n = 10$: 0.0001% against 0.0008%. The direction is systematic, which is why the negative binomial model — exact under the stated assumptions — replaced it.

q	n=1	2	3	4	5	6	7	8	9	10
2%	4.000	0.237	0.016	0.0011	$8\cdot10^{-5}$			$< 10^{-5}$		
5%	10.000	1.450	0.232	0.039	0.0066	0.0012	$2.1\cdot10^{-4}$	$4\cdot10^{-5}$	$< 10^{-5}$	
10%	20.000	5.600	1.712	0.546	0.178	0.059	0.020	0.0067	0.0023	0.0008
20%	40.000	20.800	11.584	6.669	3.916	2.331	1.401	0.848	0.516	0.316
30%	60.000	43.200	32.616	25.207	19.762	15.645	12.475	10.003	8.055	6.511
40%	80.000	70.400	63.488	57.958	53.314	49.300	45.769	42.621	39.787	37.218
45%	90.000	85.050	81.375	78.342	75.716	73.375	71.252	69.299	67.487	65.793
48%	96.000	94.003	92.508	91.264	90.177	89.201	88.307	87.478	86.703	85.972
50%					100 for every n					

Table 2: The double-spend success probability $r(q, n)$, in percent, computed by script from Theorem 5.2. On the rows the paper's Table 1 also prints ($q \in \{2, 10, 20, 30, 40, 48, 50\}$%), the script reproduces its values cell-for-cell; the 5% and 45% rows are this volume's additions.

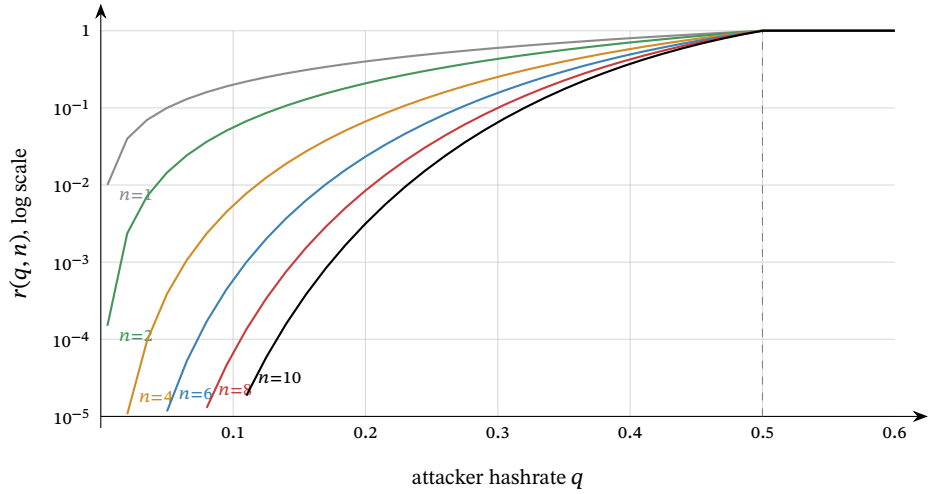

Figure 3: The success probability $r(q, n)$ against the attacker's hashrate share q, for $n \in \{1, 2, 4, 6, 8, 10\}$ confirmations, logarithmic vertical scale clipped at 10^{-5}; coordinates computed by script from Theorem 5.2. Every curve reaches 1 at $q = \frac{1}{2}$ (dashed) and stays there: beyond the majority line, confirmations are worthless. Below it, each added confirmation drops r by a roughly constant factor — the curves are near-equally spaced in log scale — and the factor improves as q falls.

6 What the numbers say

Theorem 5.2 compresses the security of Nakamoto consensus into one two-parameter function, and its qualitative lessons are all visible in Table 2 and Figure 3. This section states them precisely, then instantiates the one lesson that is specific to Zcash.

q	$r < 10\%$	$r < 1\%$	$r < 0.1\%$
2%	1	2	3
5%	2	3	4
8%	2	3	5
10%	2	4	6
15%	3	6	9
20%	4	8	13
25%	5	12	20
30%	9	20	32
35%	15	36	58
40%	34	82	133
45%	135	331	539

Table 3: The least number of confirmations pushing $r(q, n)$ below three targets, computed by script. The growth is logarithmic in the target but explosive in q: a 10% attacker is answered by 6 confirmations, a 45% attacker by 539 — more than eleven hours of Zcash blocks — and at 50% by no number at all.

Proposition 6.1 (No finite count is final)**.** *For every $0 < q < p$ and every n, $r(q, n) \geq 2q^n > 0$.*

Proof. By Theorem 5.2, $r = 2\Pr[A_n]$, and one way for the attacker to win the first-to-n race is to find the first n blocks outright, an event of probability q^n. □

Proposition 6.2 (Divergence at the majority line)**.** *Fix a target $\varepsilon < 1$ and let $n_\varepsilon(q)$ be the least n with $r(q, n) < \varepsilon$. Then $n_\varepsilon(q) \to \infty$ as $q \uparrow \frac{1}{2}$.*

Proof. For fixed n the function $q \mapsto r(q, n)$ is a polynomial on $[0, \frac{1}{2}]$, hence continuous, and $r(\frac{1}{2}, n) = 1$: at $q = p$ each half of the race of Remark 5.3 has probability exactly $\frac{1}{2}$ by Lemma 3.6 and symmetry. So for every n there is a $\delta_n > 0$ with $r(q, n) > \varepsilon$ whenever $q > \frac{1}{2} - \delta_n$; no fixed n survives arbitrarily close to the line. □

Remark 6.3 (Nothing is special about six)**.** The paper (§5): "There is nothing special about the default, often-cited figure of 6 confirmations. It was chosen based on the assumption that an attacker is unlikely to amass more than 10% of the hashrate, and that a negligible risk of less than 0.1% is acceptable. Both these figures are arbitrary" — six confirmations are "overkill for casual attackers, and at the same time powerless against more dedicated attackers". Table 3 is the honest replacement: pick the adversary you defend against and the risk you accept, and read off n; the pair (q, ε) is a policy, not a law of nature.

6.1 Zcash time: 75-second blocks

Corollary 4.7 says security is purchased in blocks, so the wall-clock price of a confirmation count is set by the target spacing — for Zcash, 75 seconds since the Blossom upgrade (ZIP-208; PostBlossomPoWTargetSpacing, specification §5.3), against Bitcoin's 600. Thirty minutes of waiting therefore buys 3 Bitcoin confirmations but 24 Zcash confirmations, and against a 10% adversary the difference is $r = 1.712\%$ against $r \approx 3.2 \times 10^{-12}$ (script-computed): nine orders of magnitude, for the same patience. ZIP-208 makes precisely this argument for the spacing change — "the number of confirmations needed increases more slowly than the decrease in block time".

The comparison holds q fixed, and that is its honest limit: the model contains no network, so nothing in it prices the disadvantage faster blocks impose on the *honest* side, whose blocks are found

closer together and are therefore likelier to race one another and self-orphan. At 75 seconds against propagation times of fractions of a second the effect is small, but it is a modelling gap, recorded next.

Remark 6.4 (Where the model bends)**.** Three assumptions do not survive contact with the deployed chain, and none of them changes the shape of the results. (i) *Difficulty is not constant*: Zcash retunes the target after every block (Section 8.2); over an attack lasting tens of blocks the drift is small, and the race is scored in work, not block counts, so the analysis carries over with q read as the attacker's share of *work* production — up to a granularity caveat quantified in Section 8.2. (ii) *Hashrate is not constant*: q in the theorems is whatever share the attacker sustains for the attack's duration; the paper's own caveat is that T_0 re-enters only if the attacker cannot sustain his hashrate long enough, which it judges unlikely for a serious attacker. (iii) *Propagation is not instantaneous*: honest self-orphaning wastes a sliver of honest work, slightly raising the attacker's effective share; the deployed spacing keeps the sliver thin. A fourth gap is not the model's: the attacker of this volume follows the protocol's validity rules perfectly and attacks only the *choice* between valid histories. Attacks on the rules themselves are other volumes' subjects.

7 The economics of the attack

The probabilities of Section 5 answer the adversarial merchant, who assumes the customer is an attacker. The paper's §6 answers the realistic one: when is the attack worth mounting at all? Throughout this section $q < p$ — "[o]therwise, all bets are off" (§6) — and the model is deliberately stylised; its own caveats close the section.

The attacker targets k merchants simultaneously with payments of v each, all invalidated by the same secret branch; the goods are worth αv to him, $0 < \alpha \leq 1$; he gives up after mining o blocks in vain; and each of his blocks would earn the block income B if it ended up in the accepted chain. If the attack succeeds his blocks are all accepted (he keeps goods *and* coins *and* rewards); if it fails he has paid kv, keeps goods worth $k\alpha v$, and his o orphaned blocks forfeit oB.

Proposition 7.1 (Profitability; Rosenfeld §6)**.** *The attacker's expected profit over not attacking is*

$$k\alpha v - (1-r)(kv + oB) \;=\; kv(\alpha + r - 1) - (1-r)\,oB,$$

positive if and only if

$$v \;>\; \frac{(1-r)\,oB}{k(\alpha + r - 1)}$$

(when $\alpha + r > 1$; when $\alpha + r \leq 1$ the attack never profits). A merchant is therefore safe whenever

$$v \;\leq\; \frac{o(1-r)B}{k(\alpha + r - 1)} \;\overset{\alpha=1}{=}\; \frac{o}{k} \cdot \frac{B(1-r)}{r}, \tag{3}$$

which with the paper's choices $o = 20$, $k = 5$, $\alpha = 1$, $B = 25$ BTC is its equation (2), $v \leq 100(1/r - 1)$ BTC.

Proof. The gain $k\alpha v$ is certain (the goods are obtained either way); the loss $kv + oB$ is paid exactly when the attack fails, with probability $1 - r$. Rearranging the positivity condition gives the threshold; substituting the paper's constants gives $20 \cdot 25 \cdot (1-r)/(5r) = 100(1-r)/r$. □

7.1 Instantiation in ZEC

The block income that an orphaned block forfeits is what the miner would have collected: the miner subsidy plus fees. At the current mainnet height the block subsidy is 1.5625 ZEC (156,250,000 zatoshi, specification §7.8; `zebra-chain/src/parameters/network/subsidy.rs:447`), of which the two

active funding streams take 20% — 8% to the Zcash Community Grants stream and 12% to the coinholder-controlled fund, both running to the third halving at height 4,406,400 (specification §7.10.1; ZIP-1016; `subsidy.rs:338`) — leaving the miner

$$B \;=\; 1.25\ \text{ZEC} + \text{fees}$$

per block (`miner_subsidy`, `subsidy.rs:480`). The funding-stream portion is not the attacker's under either outcome, so $B = 1.25$ ZEC is the right forfeiture figure (fees, small on today's chain, only raise it). Table 4 evaluates bound (3) with the paper's $o = 20$, $k = 5$, $\alpha = 1$.

q	n=1	2	4	6	8	10
5%	45	339	12,909	430,929	13,664,382	420,922,251
10%	20	84	911	8,449	74,344	636,146
20%	7	19	69	209	584	1,578
30%	3	6	14	26	44	71
40%	1	2	3	5	6	8

Table 4: The maximal safe transaction value in ZEC, $\lfloor 4B(1-r)/r \rfloor$ with $B = 1.25$ ZEC, computed by script; the analogue of the paper's Table 2, which used $B = 25$ BTC. Values are floored and inherit every assumption of the model; the caveats of Remark 7.3 apply in full.

The table's message survives its assumptions. Against small attackers a handful of confirmations protects any plausible payment; against a 30% attacker, six confirmations protect only a payment of about 26 ZEC, and a merchant accepting more should wait longer — the required count grows only logarithmically in the value at stake, by the geometric decay of Theorem 5.2.

Remark 7.2 (Majority is a different problem)**.** When $q \geq p$ the economics change character entirely: the paper notes a majority attacker can double-spend at no cost beyond normal mining, reject every other miner's blocks to take the whole issuance, and exclude transactions at will, driving honest miners out and entrenching himself — so a majority attack "is best seen as an attempt to destroy" the currency, motivated from outside it (a short position, a competing system), not an attempt to profit inside it. Confirmation policy is not the defence at that point; the defence is the cost of assembling the majority, which lies outside this model.

Remark 7.3 (The model's own caveats)**.** The paper flags each stylisation itself. The give-up point $o = 20$ "is a significant simplification" (an optimal attacker balances completion against compounding losses); the safe values "should be taken with a grain of salt, because of the many modeling assumptions"; and a model resting on mining *costs* rather than forfeited *rewards* would make the required confirmations linear, not logarithmic, in the transaction value — "very poor security, hence in a situation where this is relevant, we have already lost anyway" (§6). The bound's role is the shape it reveals — logarithmic patience buys exponential safety — not its third significant digit.

8 The deployed rule and its safety rails

The model of Sections 4–7 is clean; the deployment is not. This section walks the distance between them: how the best chain is actually chosen, how difficulty actually moves, how well Equihash fits the memoryless model, and the node-local rails — reorg windows, maturity, wallet policy — that bound the theory's tail risks in practice.

8.1 Chain selection in code

In zebra, candidate chains live in the non-finalized state as a set ordered by `impl Ord for Chain` (`zebra-state/src/service/non_finalized_state/chain.rs:2623--2696`); the best chain

is the set's maximum. The ordering's doc comment is exact about its two criteria: "Chains with higher cumulative Proof of Work are [`Ordering::Greater`], breaking ties using the tip block hash." Cumulative work is the sum of Definition 2.1 over the chain's blocks (`chain.rs:1803--1809`).

Remark 8.1 (The tie-break nobody implements)**.** The specification's tie-break is first-seen (Definition 2.2), and it is also Rosenfeld's ("the one of which the node learnt first"). The retired zcashd implemented it: its comparator sorted by `nChainWork`, then by `nSequenceId` — "[s]equential id assigned to distinguish order in which blocks are received" (`src/main.cpp:194--213`, `src/chain.h:363--364`) — so at equal work the incumbent tip stayed. Zebra does not: blocks are downloaded in parallel and arrival times are not tracked, so it breaks work ties by the larger tip hash, and its own doc comment calls this a "departure from the consensus rules" that "may delay network convergence, for as long as the greater hash belongs to the later mined block" (`chain.rs:2629--2636`). With zcashd retired, the specification's first-seen rule is implemented by no running validator. For this volume's analysis the divergence is immaterial: ties are transient states broken by the next block (Figure 1), and no theorem above depends on how a node sits out a tie.

8.2 Difficulty in motion

Assumption 4.2 freezes difficulty; Zcash does not. The specification (§7.7.3) adopts "a difficulty adjustment algorithm based on DigiShield v3/v4", and — "[u]nlike Bitcoin, the difficulty adjustment occurs after every block". Each block's target is recomputed from the mean target of the last $\mathsf{PoWAveragingWindow} = 17$ blocks and the actual timespan of that window measured between medians of $\mathsf{PoWMedianBlockSpan} = 11$ timestamps; the timespan is dampened towards nominal by a factor of $\mathsf{PoWDampingFactor} = 4$ and clamped to $[\lfloor \mathsf{AWT}(1 - 16\%) \rfloor, \lfloor \mathsf{AWT}(1 + 32\%) \rfloor]$ ($\mathsf{PoWMaxAdjustUp} = 16\%$, $\mathsf{PoWMaxAdjustDown} = 32\%$, $\mathsf{AWT} = 17 \times 75$ s), with the target capped at $\mathsf{PoWLimit} = 2^{243} - 1$ (all constants: specification §5.3; `zebra-state/src/service/check/difficulty.rs`).

Remark 8.2 (Why the race survives per-block retuning)**.** Three observations keep Theorem 5.2 meaningful on the deployed chain. First, the adjustment is slow relative to an attack: its window is 17 blocks ($\approx$ 21 minutes) and per-step movement is dampened fourfold and clamped, so over the tens of blocks a typical race lasts, both branches mine at nearly the difficulty prevailing at the fork unless a side works to move its own. Secondly, the secret branch computes its own schedule: the adjustment "use[s] only information from blocks preceding the given block height" (§7.7.3), so the attacker's chain retunes from the attacker's own timestamps, subject to the timestamp rules — `nTime` strictly above the median of the preceding 11, at most 90×60 seconds above it (§7.6), and at most two hours ahead of a validator's clock on receipt (a non-consensus check). Thirdly — and here the model genuinely bends — the comparison is by summed *work*, and work per block moves inversely with the target (Definition 2.1). An attacker who stretches his timestamps lowers his own difficulty and mints more, lighter blocks from the same expected hashing; compressing them does the opposite. The exchange preserves his expected work *production*, not his odds: a first-passage race depends on the granularity of its steps as well as their drift, and the same expected work packed into fewer, heavier blocks raises an underdog's chance of ever drawing strictly ahead — lumpy progress favours the trailing side. Script-simulated at $q = 0.3$ from a three-work-unit deficit: with equal blocks the overtake probability is 0.034 (matching the analytic $(q/p)^4$), with double-work blocks 0.10, with quadruple-work blocks 0.22. Stretched timestamps therefore *hurt* the attacker; the profitable direction is compression — raising his own difficulty within the timestamp rules — and that advantage is real, is not priced by Assumption 4.2, and is bounded by the damped, clamped per-block adjustment over a race's length. The tables of this volume are the equal-weight baseline, and the reading of q as the attacker's share of work production holds while both branches' per-block work stays close — which the first observation grants at the start of a race, and the clamp enforces over its ordinary length.

8.3 Equihash and the memoryless model

Zcash's proof of work is Equihash with parameters $n = 200$, $k = 9$ (specification §7.7, §7.7.1; the algorithm is Biryukov and Khovratovich's, ePrint 2015/946), but the quantity the race counts is gated one step later: "[t]he difficulty filter is unchanged from Bitcoin, and is calculated using SHA-256d on the whole block header (including `solutionSize` and `solution`)", required to be at most `ToTarget(nBits)` (§7.7.2). Each candidate Equihash solution therefore faces an independent, fresh SHA-256d lottery, exactly the Bernoulli trial of Remark 3.7.

The specification nowhere asserts progress-freeness — the argument is this volume's, not a citation. What Equihash changes is the granularity: trials arrive in batches, one batch per memory-hard solver run, each run yielding a Poisson-distributed handful of candidate solutions. Progress within a run is discarded when a competing block arrives, so the memoryless approximation is exact between runs and coarse within one; the approximation is good exactly because a solver run (seconds) is short against the 75-second spacing. A proof-of-work whose solve time were comparable to the spacing would make block discovery visibly non-exponential and pull the attribution sequence away from Proposition 3.4; Zcash's parameters keep the gap wide.

8.4 Reorg rails, maturity, and the finality floor

Nothing in consensus caps how deep a reorganisation may reach — by Proposition 6.1 no depth is truly final — but both validator implementations bolted rails onto the theory, and the specification acknowledges them: "A full validator MAY impose a limit on the number of blocks it will 'roll back' when switching from one best valid block chain to another that is not a descendent. For `zcashd` and `zebra` this limit is 100 blocks" (§3.3). Neither half of that sentence matches deployment.

The retired zcashd enforced `MAX_REORG_LENGTH = COINBASE_MATURITY − 1 = 99` (`src/main.h:66`): a would-be reorganisation deeper than 99 blocks did not fail over to the heavier chain — the node refused and halted, logging "[a] block chain reorganization has been detected that would roll back %d blocks! This is larger than the maximum of %d blocks, and so the node is shutting down for your safety" (`src/main.cpp:4331--4350`).

The deployed zebra keeps a rolling window of `MAX_BLOCK_REORG_HEIGHT = 1000` blocks (`zebra-chain/src/parameters/constants.rs:30`): while the best chain exceeds that length its root blocks are committed to the finalized state (`zebra-state/src/service/write.rs:455--467`), and any later block at or below the finalized tip is rejected as an `OrphanedBlock` (`zebra-state/src/service/check.rs:224--238`) — a fork rooted below the window cannot be followed at all. The constant's own documentation is careful about its standing: "[t]his is a local-only node policy; it is not part of consensus. The window is sized as a defence-in-depth measure against sustained consensus splits."

Remark 8.3 (A three-way divergence, and what the rails buy)**.** The specification says 100 for both validators; zcashd enforced 99; zebra enforces 1000 — and flags the gap itself, quoting the specification's 100 next to its own constant (`zebra-state/src/request.rs:819--827`). ZIP-307's security model inherits the stale figure ("no full Zcash node will roll back the chain by more than 100 blocks"), which no longer describes the sole running validator; the *Sync Guide* tracks that thread (§"Reorganisation detection and recovery"). For this volume's subject the rails change nothing a merchant sees: every confirmation count in Tables 2 and 3 sits far inside a 1000-block window (≈ 21 hours), so $r(q, n)$ governs commerce exactly as derived. What the rails cap is the catastrophe tail — a secret branch deeper than the window meets a wall of nodes that will not follow it automatically, converting a consensus rewrite into a manual-intervention event. The specification adds one more floor, socially anchored: a full validator that risks Mainnet funds or displays transaction information "MUST do so only for a block chain that includes the activation block of the most recent settled network upgrade" (§3.3) — history behind a settled upgrade's activation is not up for probabilistic debate at all.

The protocol itself hard-codes one confirmation count. A *transparent* output is the Bitcoin-inherited kind, its value and destination in cleartext on chain. "A transaction MUST NOT spend a

transparent output of a coinbase transaction from a block less than 100 blocks prior to the spend" (§7.1.2; `MIN_TRANSPARENT_COINBASE_MATURITY` = 100, `zebra-chain/src/transparent.rs:54`) — miners wait $n = 100$ on their own income. Read through Theorem 5.2 (script-computed): $r(0.3, 100) \approx 3.7 \times 10^{-9}$ and $r(0.4, 100) \approx 0.43\%$, but $r(0.45, 100) \approx 15.7\%$ — even the deepest confirmation rule in the protocol is strong at moderate q and porous near the majority line, as Proposition 6.2 says every fixed count must be. Since the Heartwood network upgrade (ZIP-213) a coinbase may shield its outputs, and the maturity rule binds only the transparent ones.

8.5 What wallets actually wait for

Deployed confirmation policy lives in librustzcash. Structure `ConfirmationsPolicy` defaults to 3 confirmations for *trusted* outputs (change, and notes from senders the wallet trusts) and 10 for *untrusted* ones, with zero-confirmation shielding of transparent funds allowed (`zcash_client_backend/src/data_api/wallet.rs:444--455`, citing the draft ZIP-315); the spend anchor — the note-commitment-tree root against which a spend proves its note exists (*Crypto Guide*, §"Privacy-preserving membership via zero-knowledge paths") — sits 3 blocks below the target height (`wallet.rs:574--576`). The Android SDK passes exactly these defaults for transfers, and for shielding the minimum policy, under which the transparent funds being shielded need no confirmations at all (`backend-lib/src/main/rust/lib.rs:2096`, `:2158`, `:2192`); its UI reports a transaction *confirmed* at 10 confirmations (`TransactionOverview.kt:89`). The policy layer above these constants is the *Wallet Guide*'s subject (§"Confirmations, trust, and spendability").

Table 2 prices the defaults. Three confirmations hold a 10% adversary to $r = 1.712\%$ — acceptable for outputs the wallet already trusts — while ten hold the same adversary to 0.0008% and a 20% adversary to 0.316%. In the model's terms, the trusted/untrusted split is a bet that senders one transacts with repeatedly are not mounting hashrate attacks, and the 10-row is the posture towards everyone else. Both rows, like every fixed count, are a (q, ε) policy in the sense of Remark 6.3, not a guarantee.

9 Relation to the rest of the series

This volume opens the deployed-protocol group of the series — everything above it takes the ledger constructed here for granted — and three threads run outward from it.

Downward-facing trust. A light client never evaluates Definition 2.2 itself: it outsources the chain view to a server and trusts "that proxy to provide a correct view of the best chain" (*Sync Guide*, §"Architecture and authority"). Every confirmation count such a client displays is counted *inside* the served view; the probabilities of this volume apply to it only insofar as the view is honest. Succinctly verifying that a served view really is the most-work chain — without downloading it — is exactly the problem the *FlyClient Guide* treats (§"The problem: verifying a chain without downloading it"), and its starting assumption is this volume's conclusion: that the most-work chain is the one the network converges on.

The gap finality closes. Proposition 6.2 is the honest limit of Nakamoto consensus: as the adversary approaches half the work, no confirmation count survives. Crosslink's answer is to stop paying for the tail probabilistically — a vote among a fixed validator committee, safe while fewer than a third of it misbehaves, checkpoints the proof-of-work chain, and "final" becomes an assured status rather than a shrinking r (*Crosslink Guide*, §"The ebb-and-flow model", §"Finality as a status"). The present volume supplies the curve that design is buying its way off of; the 1000-block window of Remark 8.3 is the node-policy stopgap standing where assured finality would go.

What this volume owes downward. Exactly one debt: every theorem above stands on the *Math Guide*'s probability section — the definition of probability, its extension past finite spaces, and the continuity theorem the catch-up proof invokes. Upward it owes nothing: the citations to Ironwood, Wallet, Sync, FlyClient, and Crosslink are delimitations and pointers, not dependencies. The ledger the rest of the series builds on is, from here up, a probabilistic object with known constants — and the constants are Tables 2, 3, and 4.